

Correlated Subqueries to CTEs

#sql-server #performance #set-based #refactoring

What Is a Correlated Subquery

A correlated subquery references columns from the outer query, which means the database engine must evaluate the subquery once per row of the outer result set. The subquery is logically correlated to each outer row.

```
-- Correlated subquery: runs once per row of OeOrder
Select
    O.OrderId,
    O.PharmacyId,
    (Select Min(CF.WeekDayCode)
     From CfStoreDeliveryCourierCutOff CF
     Where CF.SysEndPharmacyId = O.PharmacyId      -- correlation!
           And CF.WeekDayCode >= DatePart(Weekday, GetDate())) As NextDeliveryDay
From OeOrder O
```

If the outer query returns 10,000 rows, the subquery executes 10,000 times. Each execution touches the CfStoreDeliveryCourierCutOff table independently. Even with an index, that's 10,000 separate seek operations.

Why the Optimizer Can't Always Fix This

SQL Server's optimizer is sometimes smart enough to decorrelate a subquery, converting it internally to a join. But it fails when:

- The subquery has complex logic (CASE expressions, multiple aggregations).
- Multiple correlated subqueries reference different tables.
- The correlation involves computed expressions.
- The subquery is in a CASE WHEN branch.

When decorrelation fails, you get the per-row execution pattern, and it shows up as massive logical reads in SET STATISTICS IO.

The CTE Alternative

A Common Table Expression (CTE) computes the result once as a set-based operation, then joins back:

```
;With NextDeliveryDay As
(
    Select
        SysEndPharmacyId,
        Min(Case
            When WeekDayCode >= DatePart(Weekday, GetDate())
            Then WeekDayCode
            End) As ThisWeekDay,
        Min(WeekDayCode) As FirstWeekDay
    From CfStoreDeliveryCourierCutOff With (NoLock)
    Group By SysEndPharmacyId
)
Select
    O.OrderId,
```

```

    0.PharmacyId,
    Coalesce(NDD.ThisWeekDay, NDD.FirstWeekDay) As NextDeliveryDay
From OeOrder O
Left Join NextDeliveryDay NDD On NDD.SysEndPharmacyId = O.PharmacyId

```

The key insight: the CTE scans CfStoreDeliveryCourierCutOff once, groups by pharmacy, and produces a small result set. The join back to the outer query is a simple lookup. Total reads on the delivery table: 6 logical reads instead of thousands.

Recognizing the Pattern

Look for these symptoms in existing code:

1. Scalar subqueries in SELECT list that reference outer table aliases.
2. Repeated table access. The same table appears in multiple subqueries.
3. SET STATISTICS IO shows a table with reads far exceeding its size (it is being read N times).
4. Execution plan shows Nested Loops with an Index Seek on the inner side that has an actual execution count matching the outer row count.

The Refactoring Steps

1. Identify all correlated subqueries referencing the same table or similar logic.
2. Build a single CTE that pre-aggregates the data grouped by the correlation key.
3. Use CASE expressions inside the aggregate to handle the conditional logic that was in different subqueries.
4. LEFT JOIN the CTE back to the outer query (LEFT because the original subquery returned NULL for non-matches).
5. Replace the subquery references with CTE column references.

Handling the Wrap-Around Case

A subtle bug in the delivery day pattern: what happens on a Saturday if the pharmacy only delivers Mon to Fri? The WeekDayCode >= DatePart(Weekday, GetDate()) condition returns no matches, so ThisWeekDay is NULL. The FirstWeekDay column (which is just Min(WeekDayCode) without the date filter) provides the wrap-around to next Monday.

```
Coalesce(NDD.ThisWeekDay, NDD.FirstWeekDay)
```

This is equivalent to “find the next delivery day this week; if none, wrap to the first delivery day next week.”

Real-World Example

In lsp_RxfGetListOfManualFillGroups, nine correlated subqueries all hit CfStoreDeliveryCourierCutOff, one for each piece of delivery schedule information (cutoff time, day code, courier name, and so on). Each subquery ran once per row of the ~200-row result set, producing about 1,800 individual seeks against the table. The CTE refactoring collapsed all nine into a single scan and GROUP BY, then a single LEFT JOIN back to bring the CutOff row's full set of columns. The delivery table went from thousands of reads to 6.

When NOT to Use CTEs

CTEs are not materialized in SQL Server (unlike temp tables). The optimizer may re-evaluate a CTE each time it's referenced. If you reference the same CTE in multiple places, consider materializing it into a #temp table instead. For a single reference, CTEs are ideal.

Related Concepts

- **Table Variables vs Temp Tables**: another way to pre-compute results for joins.
- **Scalar UDF Parallelism Barrier**: correlated subqueries share the per-row problem with scalar UDFs.
- **Non-SARGable Predicates**: another pattern where the optimizer can't efficiently access data.
- **LEFT JOIN OR Anti-Pattern**: another join pattern that defeats set-based optimization.